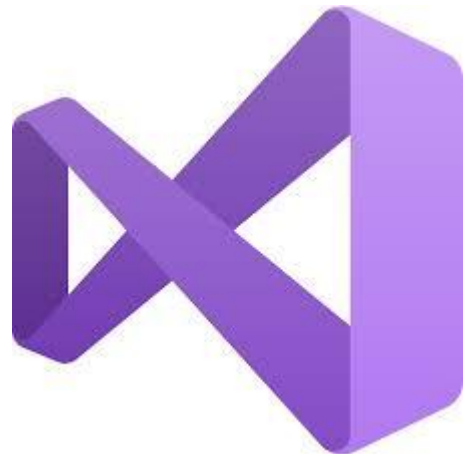




Visual Code



Visual Studio



Programmiertechnik B

GUI-Anwendungen entwickeln

Einführung .NET Framework

Visual Code

Visual Studio



Inhalt

1. .NET Framework	3
1.1. Visual Studio und .NET.....	3
1.2. Vorteile der Programmiersprachenunabhängigkeit von .NET	3
2. Visual Code	5
2.1. Beschreibung	5
2.2. Benutzeroberfläche	5
2.3. .NET Entwicklung	5
2.3.1. Voraussetzungen	5
2.4. Console Applikation erstellen	6
2.5. Debugging.....	7
2.5.1. Breakpoint	7
2.5.2. Terminaleingabe einrichten	8
2.5.3. Debugging starten.....	8
2.5.4. Release-Build-Konfiguration verwenden	10
2.6. Übersicht der Shortcuts.....	10
3. Visual Studio	11
3.1. Projekte erstellen und speichern.....	11
3.2. Einführung Entwicklungsumgebung	16
3.2.1. Erstellen Sie eine erste C#-Konsolenanwendung	16
3.2.2. Erstellen Sie eine erste C#-WPF Anwendung	17
3.2.3. Erstellen einer C#-Windows WPF Anwendung:	18



1. .NET FRAMEWORK

1.1. VISUAL STUDIO UND .NET

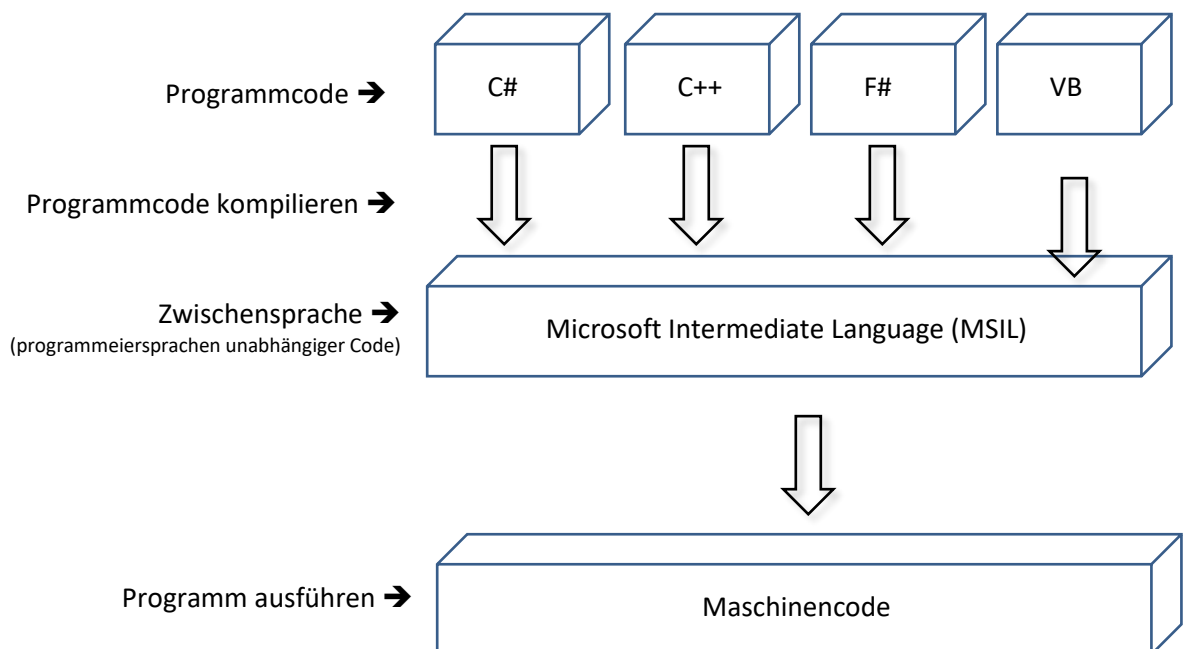
Visual Studio ist eine Entwicklungsumgebung für das .NET Framework von Microsoft.

.NET (sprich: dotnet) beschreibt eine Softwareumgebung zur Entwicklung programmiersprachen- und plattformunabhängiger Software, in dem der Programmcode verschiedener Programmiersprachen mittels eines Compilers in eine Zwischensprache übersetzt wird, die für alle Programmiersprachen gleich ist. Diese Zwischensprache ist plattformneutral und kann auf unterschiedlichen Betriebssystemen wie Windows oder Linux ausgeführt werden.

1.2. VORTEILE DER PROGRAMMIERSPRACHENUNABHÄNGIGKEIT VON .NET

Durch das Konzept der Zwischensprache sind alle .NET-Sprachen gleichwertig, auch in Bezug auf die Ausführungsgeschwindigkeit.

Entwickler können mit verschiedenen Programmiersprachen an einem Projekt arbeiten.



Programmcode erzeugen

Programmcode kann in verschiedenen .NET Programmiersprachen, wie z.B. Visual Basic, Visual C#, C++ etc. erstellt werden.

Programmcode kompilieren

Mithilfe integrierter Compiler wird der Programmcode in eine einheitliche Zwischensprache (MSIL) übersetzt.

Zwischensprache

Die Zwischensprache ist für alle .NET-Programmiersprachen gleich. Sie wird **Microsoft Intermediate Language (MSIL)** oder allgemein **Common Intermediate Language (CIL)** genannt.



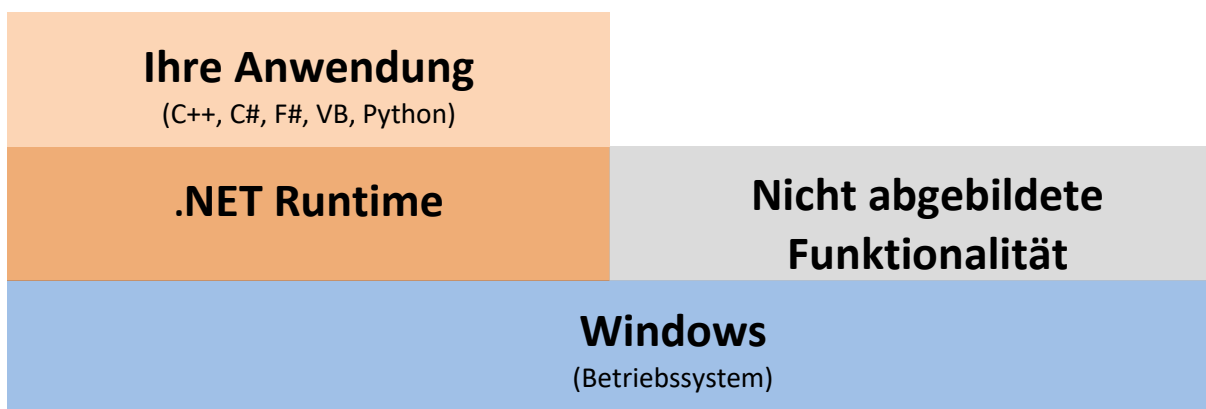
Programm ausführen

Der Code der Zwischensprache wird bei der Programmausführung nicht interpretiert, sondern durch einen JIT (Just In Time)-Compiler in den Maschinencode des entsprechenden Prozessors übersendet.

Dabei übersetzt der Compiler den benötigten Code beim ersten Programmlauf in der Reihenfolge, in der er gerade benötigt wird. Der kompilierte Maschinencode wird dann innerhalb eines Programmlaufs bei erneuten Aufrufen direkt verwendet.

.NET Plattform

Anwendung welche in z.B. in C# geschrieben werden basieren auf der Common Language Infrastructure lässt erahnen, dass Sie nicht direkt auf dem Betriebssystem kommunizieren. Dazwischen liegt die .NET Runtime bestehend aus verschiedenen Komponenten zur Ausführung der Applikationen. Sie bildet das Betriebssystem für die Applikationen ab.





2. VISUAL CODE

2.1. BESCHREIBUNG

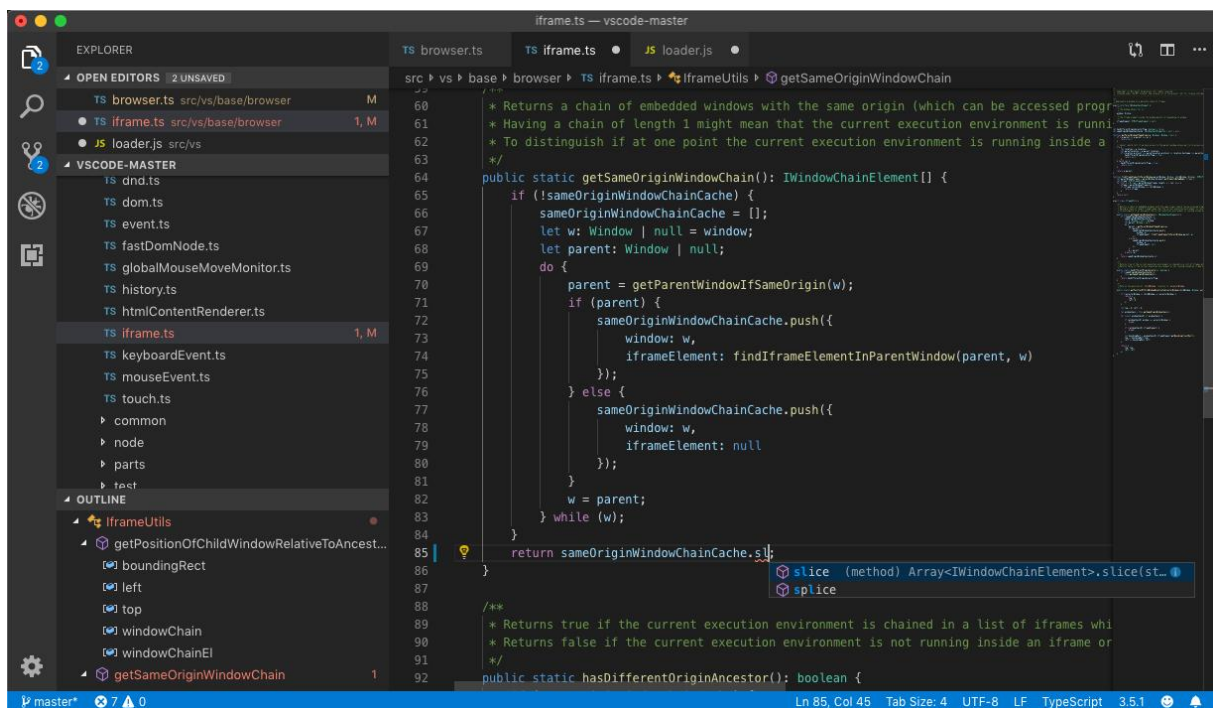
Visual Code ist ein kostenloser Quelltext Editor und steht plattformübergreifend für Windows, macOS und Linux zur Verfügung.

Visual Code unterstützt eine Vielzahl von Programmiersprachen und wurde von einem Team in der Schweiz, basierend auf dem Framework Electron entwickelt. U.a. bietet der Editor Syntaxhervorhebung, Code-Faltung, Debugging, Autovervollständigung und Versionsverwaltung.

Visual Studio Code unterstützt zudem auch die Webseiten-Erstellung mit HTML, CSS, JSON, LESS, SASS, PHP.

Über Plug-ins kann eine Unzahl von Erweiterungen (Extensions) nachinstalliert werden, womit der Funktionsumfang selektiv erweitert werden kann.

2.2. BENUTZEROBERFLÄCHE



2.3. .NET ENTWICKLUNG

2.3.1. VORAUSSETZUNGEN

- Visual Code Installation
<https://code.visualstudio.com>
- C# Erweiterung
<https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp>
- .NET 5.0 SDK
<https://dotnet.microsoft.com/download>



2.4. CONSOLE APPLIKATION ERSTELLEN

1. Starten Sie Visual Studio Code.
2. Wählen Sie im Hauptmenü Datei > Ordner öffnen (Datei > Öffnen... unter macOS).
3. Erstellen Sie im Dialogfeld "Ordner öffnen" einen Ordner "HelloWorld" und klicken Sie auf "Ordner auswählen" (Öffnen unter macOS).
4. Der Ordnername wird standardmässig zum Projektnamen und zum Namen des Namespaces. Sie werden später im Lernprogramm Code hinzufügen, der davon ausgeht, dass der Projektnamensraum HelloWorld ist.
5. Öffnen Sie das Terminal in Visual Studio Code, indem Sie im Hauptmenü Ansicht > Terminal wählen.
6. Das Terminal wird mit der Eingabeaufforderung im Ordner HelloWorld geöffnet.
7. Geben Sie im Terminal den folgenden Befehl ein:

```
dotnet new console --framework net5.0
```

Die Vorlage erstellt eine einfache "Hello World"-Anwendung. Sie ruft die Methode **Console.WriteLine(String)** auf, um "Hello World!" im Konsolenfenster anzuzeigen.

Der Code der Vorlage definiert eine Klasse, **Program**, mit einer einzigen Methode, **Main**, die ein String-Array als Argument annimmt.

```
using System;

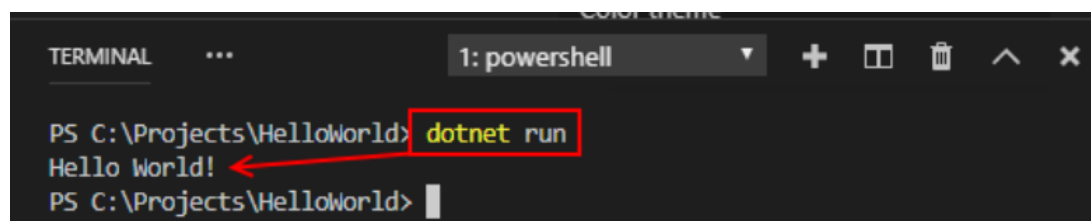
namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

Main ist der Einstiegspunkt der Anwendung, die Methode, die automatisch von der Laufzeitumgebung aufgerufen wird, wenn sie die Anwendung startet.

Alle Befehlszeilenargumente, die beim Start der Anwendung übergeben werden, sind im Array **args** verfügbar.

Die Applikation kann mit folgendem Befehl im Terminal Fenster gestartet werden:

```
dotnet run
```





2.5. DEBUGGING

Verbessern Sie die Anwendung so, dass sie den Benutzer zur Eingabe seines Namens auffordert und diesen zusammen mit Datum und Uhrzeit anzeigt.

```
Console.WriteLine("What is your name?");
var name = Console.ReadLine();
var currentDate = DateTime.Now;
Console.WriteLine($"{Environment.NewLine}Hello, {name}, on {currentDate:d} at
{currentDate:t}!");
Console.Write($"{Environment.NewLine}Press any key to exit...");
Console.ReadKey(true);
```

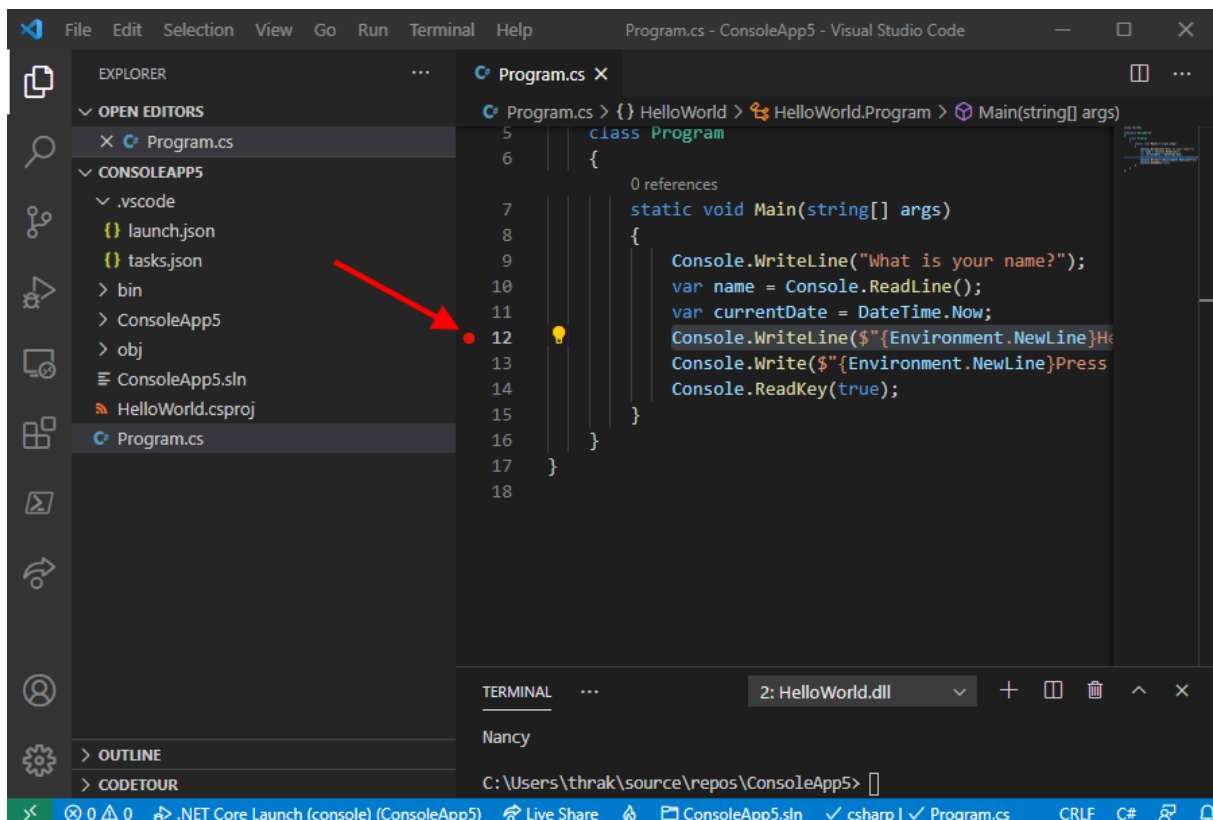
2.5.1. BREAKPOINT

Ein Haltepunkt unterbricht vorübergehend die Ausführung der Anwendung, bevor die Zeile mit dem Haltepunkt ausgeführt wird.

Öffnen Sie die Datei **Program.cs**.

Setzen Sie einen Haltepunkt in der Zeile, in der der Name, das Datum und die Uhrzeit angezeigt werden, indem Sie in den linken Rand des Codefensters klicken. Der linke Rand befindet sich links neben den Zeilennummern. Andere Möglichkeiten, einen Haltepunkt zu setzen, sind die Taste **F9** oder die Auswahl von **Run > Toggle Breakpoint** umschalten aus dem Menü, während die Codezeile ausgewählt ist.

Visual Studio Code zeigt die Zeile an, in der der Haltepunkt gesetzt ist, indem ein roter Punkt am linken Rand angezeigt wird.





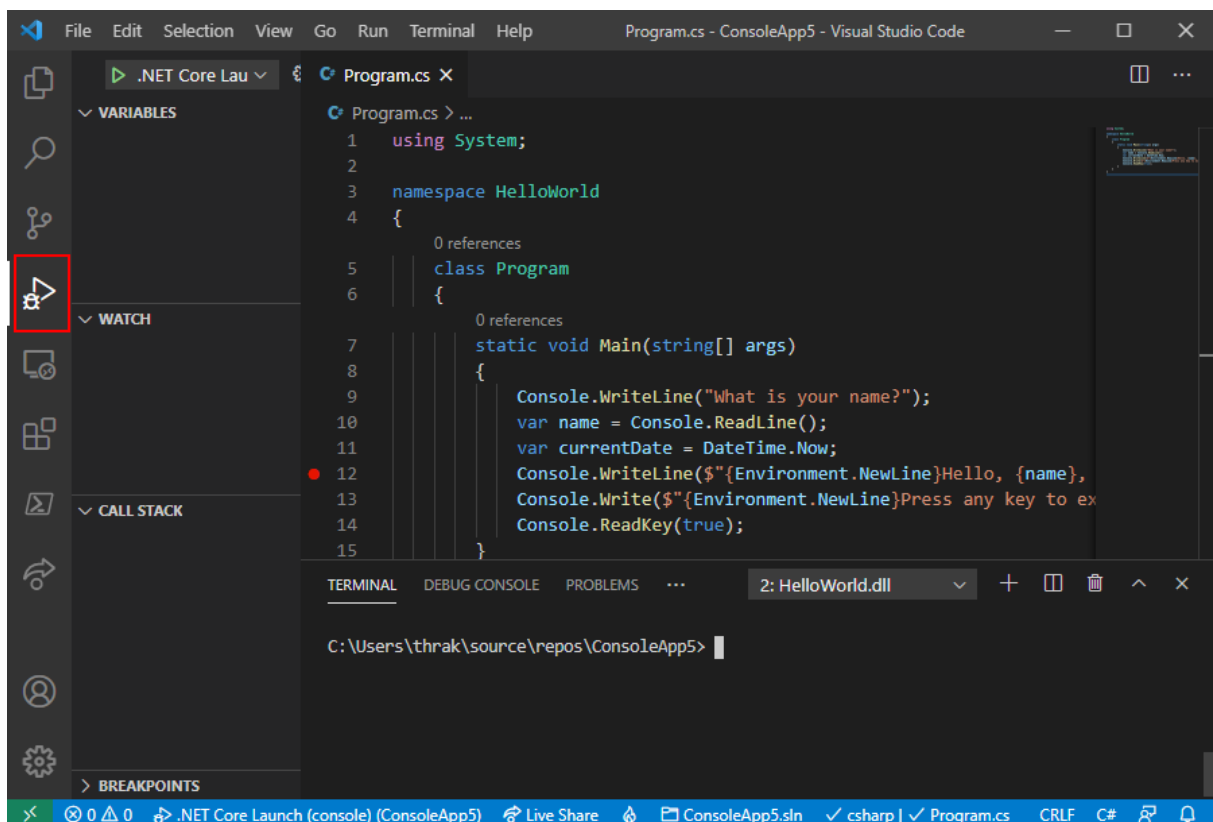
2.5.2. TERMINALEINGABE EINRICHTEN

Der Haltepunkt befindet sich nach einem Aufruf der Methode **Console.ReadLine**. Die Debug-Konsole akzeptiert keine Terminaleingabe für ein laufendes Programm. Um Terminaleingaben während des Debuggens zu verarbeiten, können Sie das integrierte Terminal (eines der Fenster von Visual Studio Code) oder ein externes Terminal verwenden. In diesem Lernprogramm verwenden Sie das integrierte Terminal.

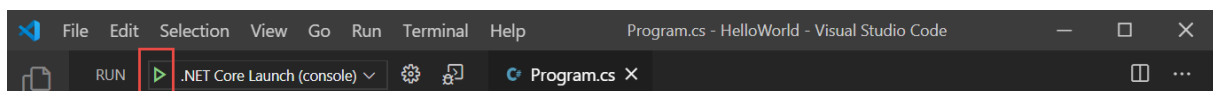
1. Datei öffnen .vscode/launch.json
2. Setting **console** von **internalConsole** nach **integratedTerminal** ändern.
`"console": "integratedTerminal",`

2.5.3. DEBUGGING STARTEN

Öffnen Sie die Debug-Ansicht, indem Sie im Menü auf der linken Seite das Symbol Debugging auswählen.

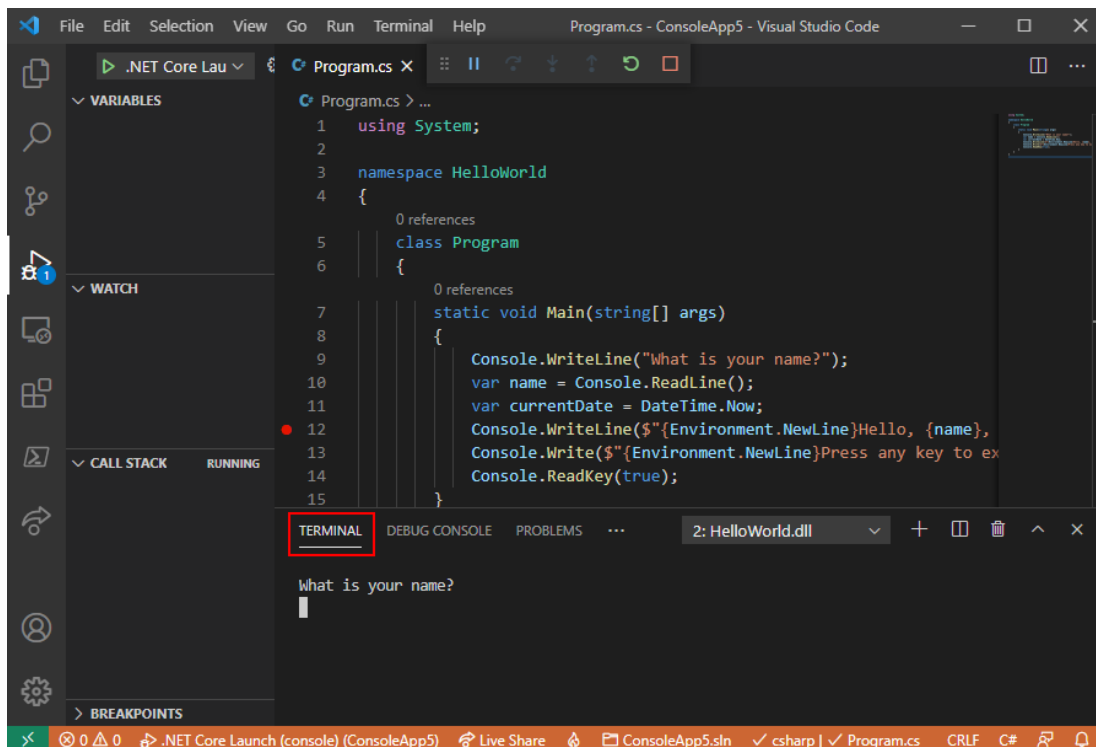


Wählen Sie den grünen Pfeil oben im Fenster neben .NET Core Launch (Konsole). Weitere Möglichkeiten, das Programm im Debugging-Modus zu starten, sind die Taste F5 oder die Auswahl von Ausführen > Debugging starten im Menü.



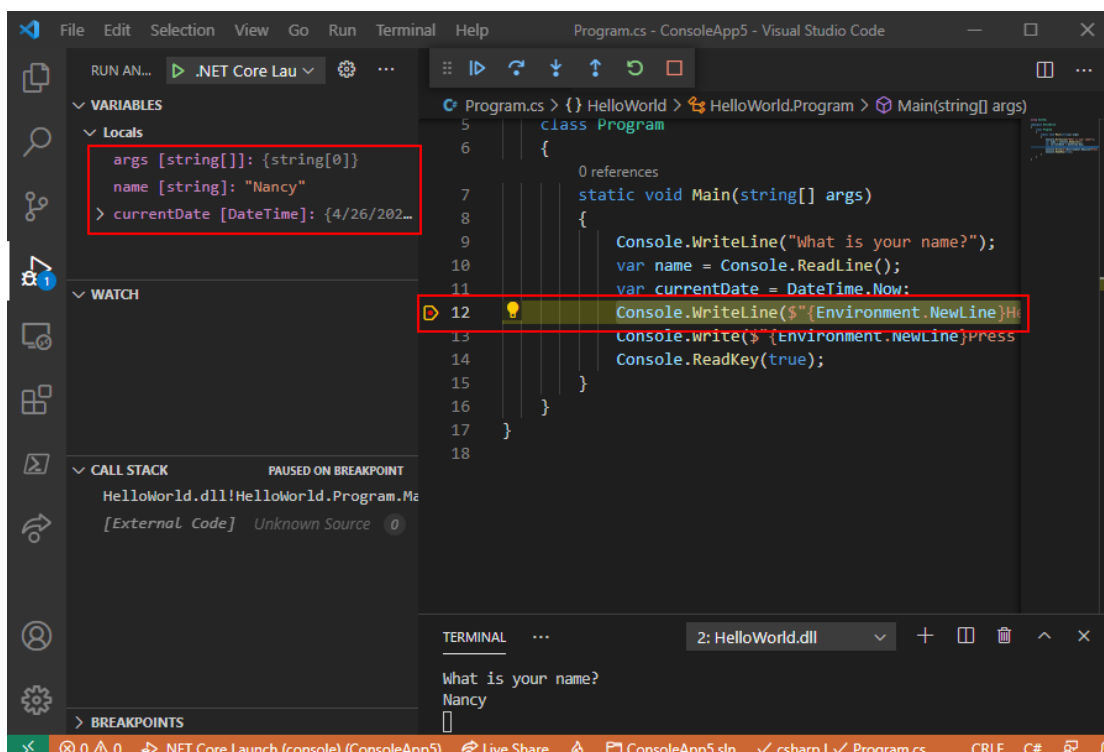


Wählen Sie die Registerkarte Terminal, um die Aufforderung "Wie lautet Ihr Name?" zu sehen, die das Programm anzeigt, bevor es auf eine Antwort wartet.



Geben Sie auf die Aufforderung zur Eingabe eines Namens eine Zeichenfolge in das Terminalfenster ein und drücken Sie die Eingabetaste.

Die Programmausführung stoppt, wenn der Haltepunkt erreicht wird und bevor die Methode **Console.WriteLine** ausgeführt wird. Im Abschnitt Locals des Fensters Variables werden die Werte der Variablen angezeigt, die in der aktuell ausgeführten Methode definiert sind.





2.5.4. RELEASE-BUILD-KONFIGURATION VERWENDEN

Sobald Sie die Debug-Version Ihrer Anwendung getestet haben, sollten Sie auch die Release-Version kompilieren und testen. Die Release-Version enthält Compiler-Optimierungen, die das Verhalten einer Anwendung beeinflussen können. So können beispielsweise Compiler-Optimierungen, die die Leistung verbessern sollen, in Multithreading-Anwendungen zu Race Conditions führen.

Um die Release-Version Ihrer Konsolenanwendung zu erstellen und zu testen, öffnen Sie das Terminal und führen Sie den folgenden Befehl aus:

```
dotnet run --configuration Release
```

2.6. ÜBERSICHT DER SHORTCUTS



Visual Studio Code

Keyboard shortcuts for Windows

General	
Ctrl+Shift+P, F1	Show Command Palette
Ctrl+P	Quick Open, Go to File...
Ctrl+Shift+N	New window/instance
Ctrl+Shift+W	Close window/instance
Ctrl+,	User Settings
Ctrl+K Ctrl+S	Keyboard Shortcuts

Basic editing	
Ctrl+X	Cut line (empty selection)
Ctrl+C	Copy line (empty selection)
Alt+ ↑ / ↓	Move line up/down
Shift+Alt+ ↑ / ↓	Copy line up/down
Ctrl+Shift+K	Delete line
Ctrl+Enter	Insert line below
Ctrl+Shift+Enter	Insert line above
Ctrl+Shift+\	Jump to matching bracket
Ctrl+] / [	Indent/outdent line
Home / End	Go to beginning/end of line
Ctrl+Home	Go to beginning of file
Ctrl+End	Go to end of file
Ctrl+↑ / ↓	Scroll line up/down
Alt+PgUp / PgDn	Scroll page up/down
Ctrl+Shift+[	Fold (collapse) region
Ctrl+Shift+]	Unfold (uncollapse) region
Ctrl+K Ctrl+[	Fold (collapse) all subregions
Ctrl+K Ctrl+]	Unfold (uncollapse) all subregions
Ctrl+K Ctrl+0	Fold (collapse) all regions
Ctrl+K Ctrl+C	Unfold (uncollapse) all regions
Ctrl+K Ctrl+U	Add line comment
Ctrl+/	Remove line comment
Shift+Alt+A	Toggle block comment
Alt+Z	Toggle word wrap

Navigation	
Ctrl+T	Show all Symbols
Ctrl+G	Go to Line...
Ctrl+P	Go to File...
Ctrl+Shift+O	Go to Symbol...
Ctrl+Shift+M	Show Problems panel
F8	Go to next error or warning
Shift+F8	Go to previous error or warning
Ctrl+Shift+Tab	Navigate editor group history
Alt+ ← / →	Go back / forward

Ctrl+M Toggle Tab moves focus

Search and replace

Ctrl+F	Find
Ctrl+H	Replace
F3 / Shift+F3	Find next/previous
Alt+Enter	Select all occurrences of Find match
Ctrl+D	Add selection to next Find match
Ctrl+K Ctrl+D	Move last selection to next Find match
Alt+C / R / W	Toggle case-sensitive / regex / whole word

Multi-cursor and selection

Alt+Click	Insert cursor
Ctrl+Alt+ ↑ / ↓	Insert cursor above / below
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Insert cursor at end of each line selected
Ctrl+L	Select current line
Ctrl+Shift+L	Select all occurrences of current selection
Ctrl+F2	Select all occurrences of current word
Shift+Alt+ →	Expand selection
Shift+Alt+ ←	Shrink selection
Shift+Alt+ (drag mouse)	Column (box) selection
Ctrl+Shift+Alt+ (arrow key)	Column (box) selection
Ctrl+Shift+Alt+PgUp/PgDn	Column (box) selection page up/down

Rich languages editing

Ctrl+Space	Trigger suggestion
Ctrl+Shift+Space	Trigger parameter hints
Shift+Alt+F	Format document
Ctrl+K Ctrl+F	Format selection
F12	Go to Definition
Alt+F12	Peek Definition
Ctrl+K F12	Open Definition to the side
Ctrl+,	Quick Fix
Shift+F12	Show References
F2	Rename Symbol
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+K M	Change file language

Editor management

Ctrl+F4, Ctrl+W	Close editor
Ctrl+K F	Close folder
Ctrl+\	Split editor
Ctrl+ 1 / 2 / 3	Focus into 1 st , 2 nd or 3 rd editor group
Ctrl+K Ctrl+ ←/→	Focus into previous/next editor group
Ctrl+Shift+PgUp / PgDn	Move editor left/right
Ctrl+K ← / →	Move active editor group

File management

Ctrl+N	New File
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+S	Save As...
Ctrl+K S	Save All
Ctrl+F4	Close
Ctrl+K Ctrl+W	Close All
Ctrl+Shift+T	Reopen closed editor
Ctrl+K Enter	Keep preview mode editor open
Ctrl+Tab	Open next
Ctrl+Shift+Tab	Open previous
Ctrl+K P	Copy path of active file
Ctrl+K R	Reveal active file in Explorer
Ctrl+K O	Show active file in new window/instance

Display

F11	Toggle full screen
Shift+Alt+O	Toggle editor layout (horizontal/vertical)
Ctrl+ = / -	Zoom in/out
Ctrl+B	Toggle Sidebar visibility
Ctrl+Shift+E	Show Explorer / Toggle focus
Ctrl+Shift+F	Show Search
Ctrl+Shift+G	Show Source Control
Ctrl+Shift+D	Show Debug
Ctrl+Shift+X	Show Extensions
Ctrl+Shift+H	Replace in files
Ctrl+Shift+J	Toggle Search details
Ctrl+Shift+U	Show Output panel
Ctrl+Shift+V	Open Markdown preview
Ctrl+K V	Open Markdown preview to the side
Ctrl+K Z	Zen Mode (Esc Esc to exit)

Debug

F9	Toggle breakpoint
F5	Start/Continue
Shift+F5	Stop
F11 / Shift+F11	Step into/out
F10	Step over
Ctrl+K Ctrl+I	Show hover

Integrated terminal

Ctrl+`	Show integrated terminal
Ctrl+Shift+`	Create new terminal
Ctrl+C	Copy selection
Ctrl+V	Paste into active terminal
Ctrl+↑ / ↓	Scroll up/down
Shift+PgUp / PgDn	Scroll page up/down
Ctrl+Home / End	Scroll to top/bottom

Other operating systems' keyboard shortcuts and additional unsigned shortcuts available at aka.ms/vscodekeybindings



3. VISUAL STUDIO

3.1. PROJEKTE ERSTELLEN UND SPEICHERN

Je nach Einsatzgebiet wird zwischen verschiedenen Projekttypen unterschieden. Folgende Projekttypen können erstellt werden:

Projekte mit grafischer Benutzeroberfläche:

- Windows-Anwendung
- WPF- und WCF Anwendungen
- WEB-Anwendungen (Server und Client)
- Erweiterungen für Office-Produkte

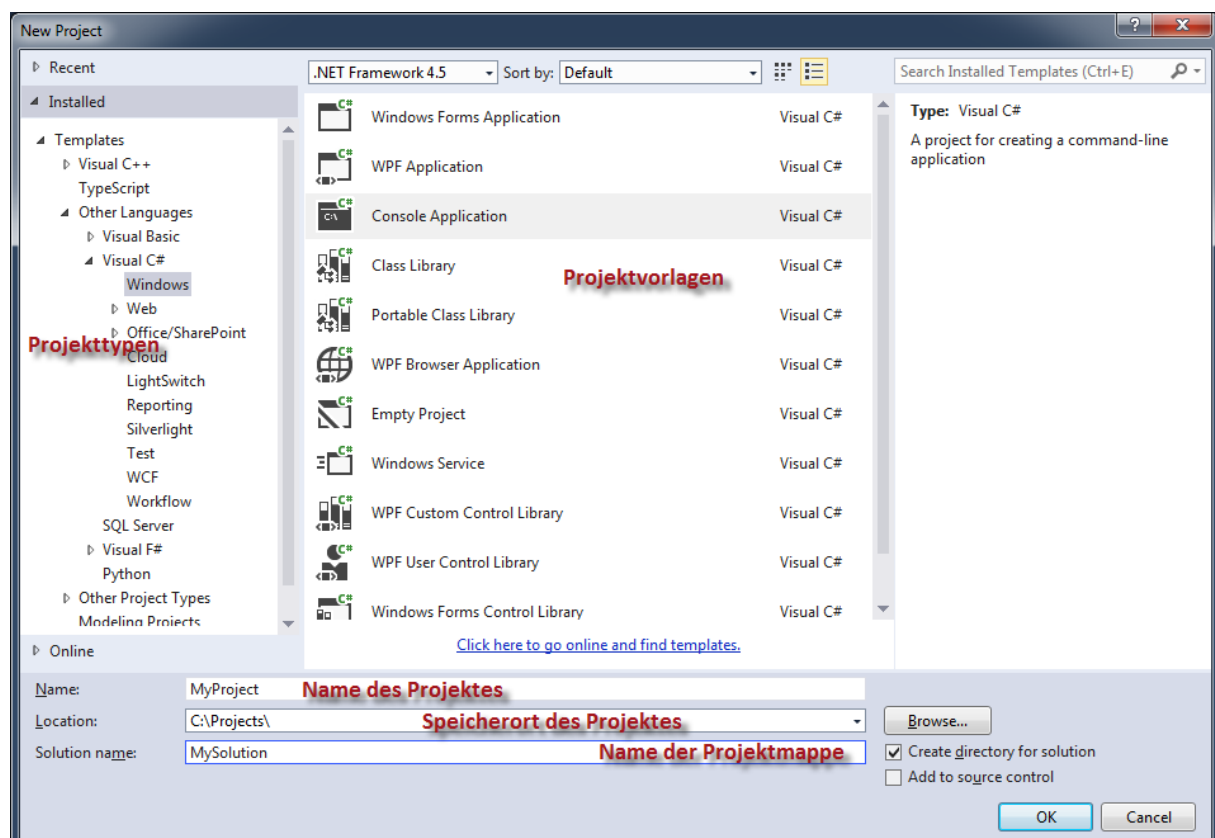
Projekte ohne grafische Benutzeroberfläche:

- Konsole Anwendung
- Windows-Dienste

Programmkomponenten und sonstige Projektarten

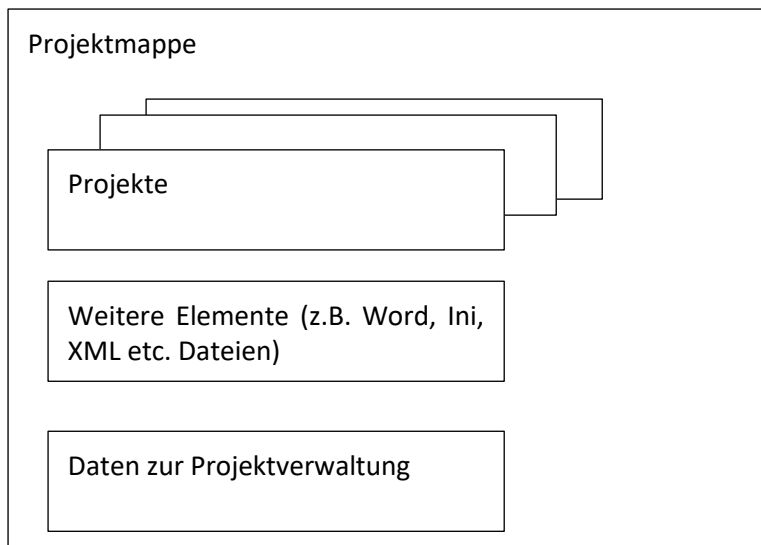
- Klassenbibliotheken
- Neue Steuerelemente
- Etc.

Rufen Sie den Menüpunkt DATEI → NEU → PROJEKT auf:

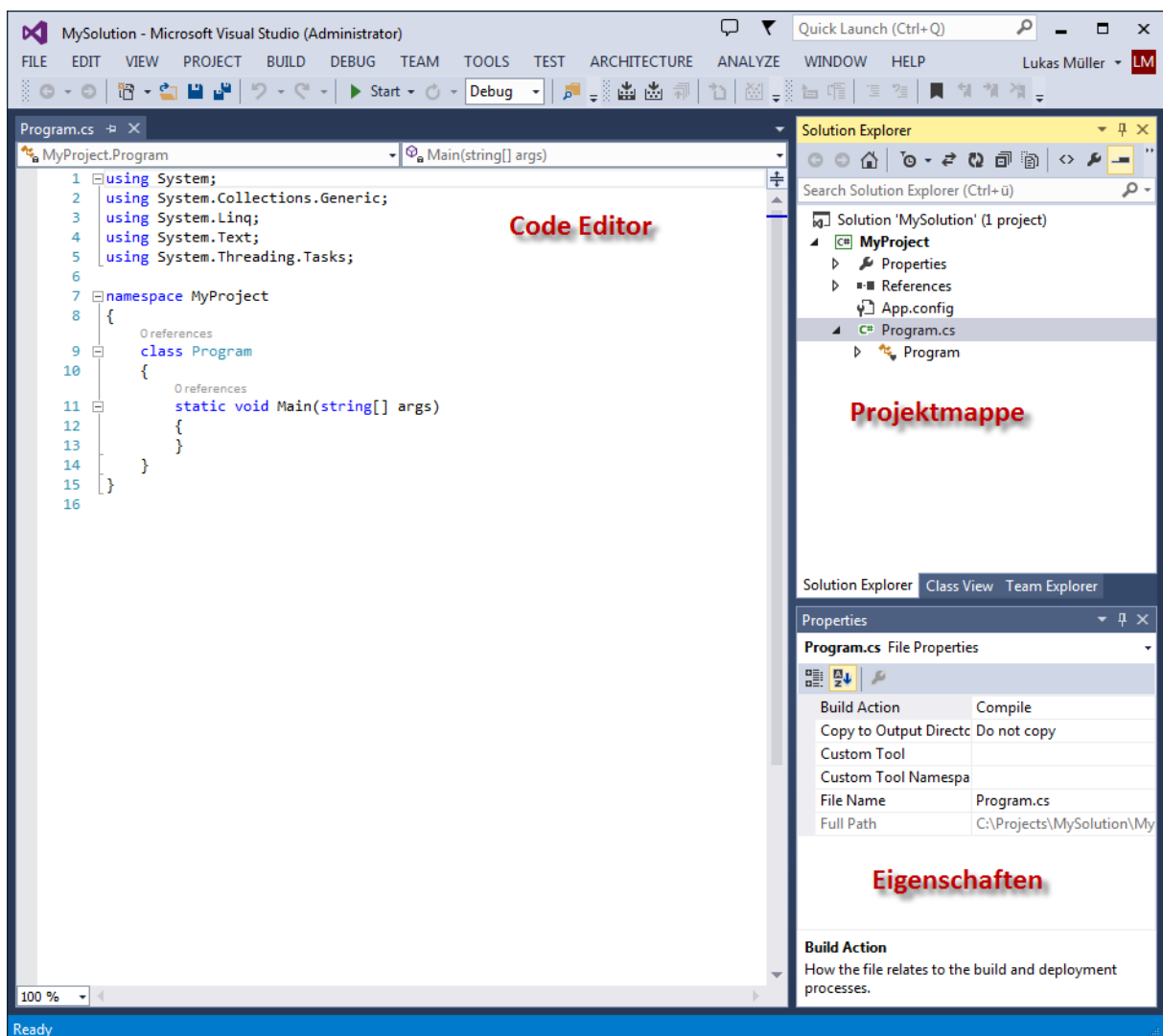




Projekte in Projektmappen organisieren

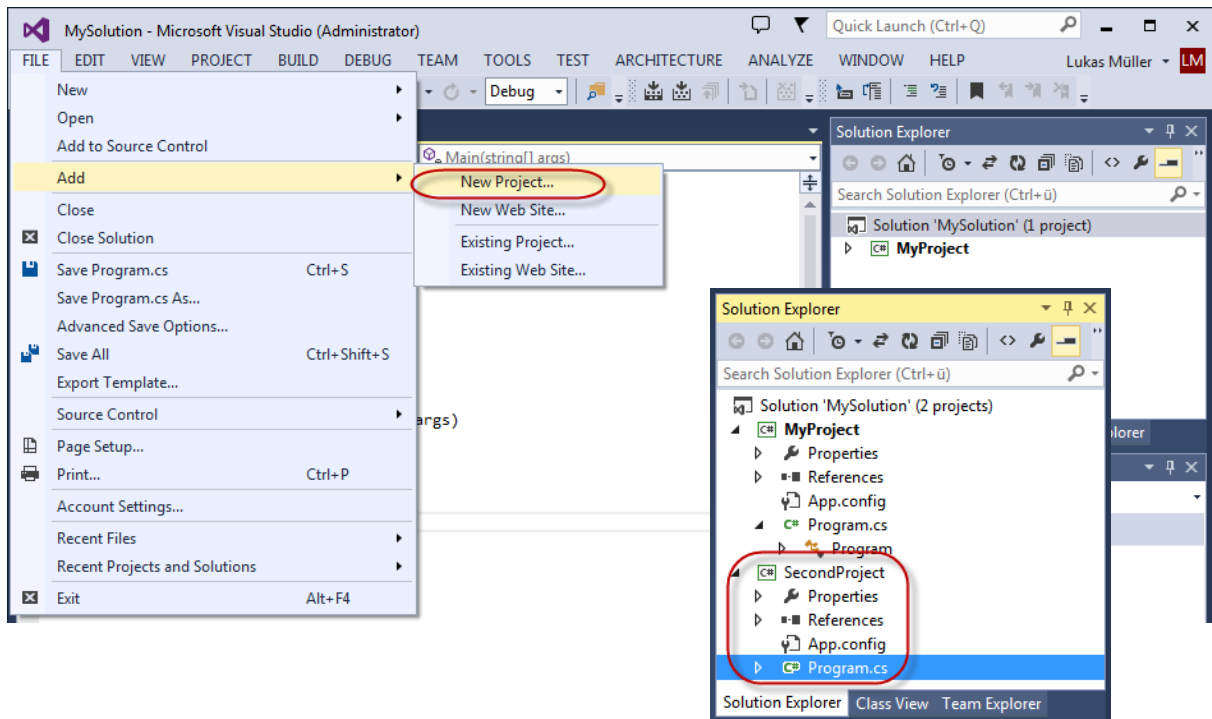


Die Anordnung der verschiedenen Fenster kann individuell nach eigenem Gutbefinden angeordnet werden. Wichtig ist dabei, dass die oft benutzen Einstellungen immer direkt eingeblendet und im Zugriff stehen.

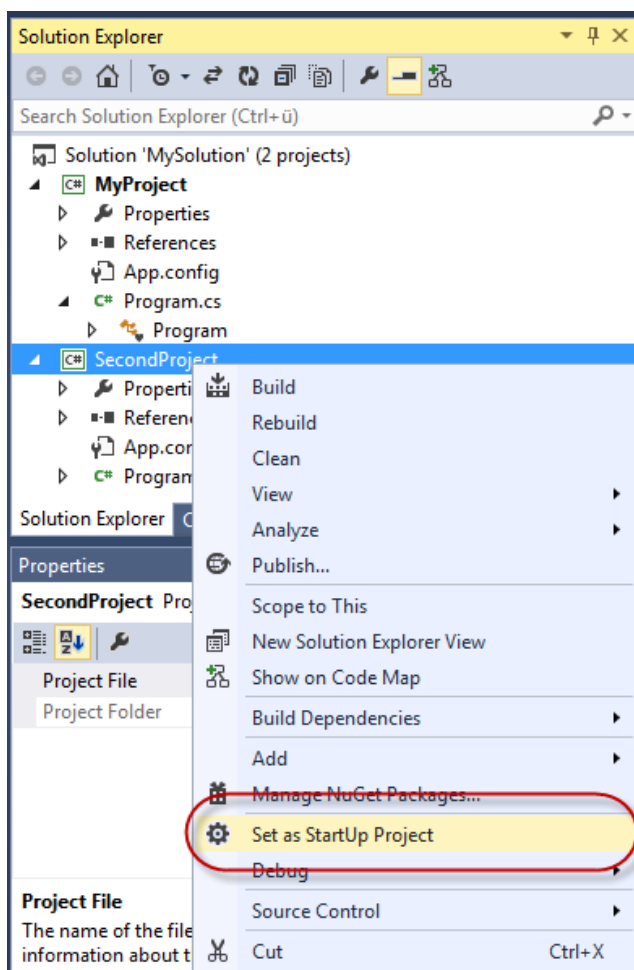




Einer Projektmappe können weitere Projekte zugefügt werden.

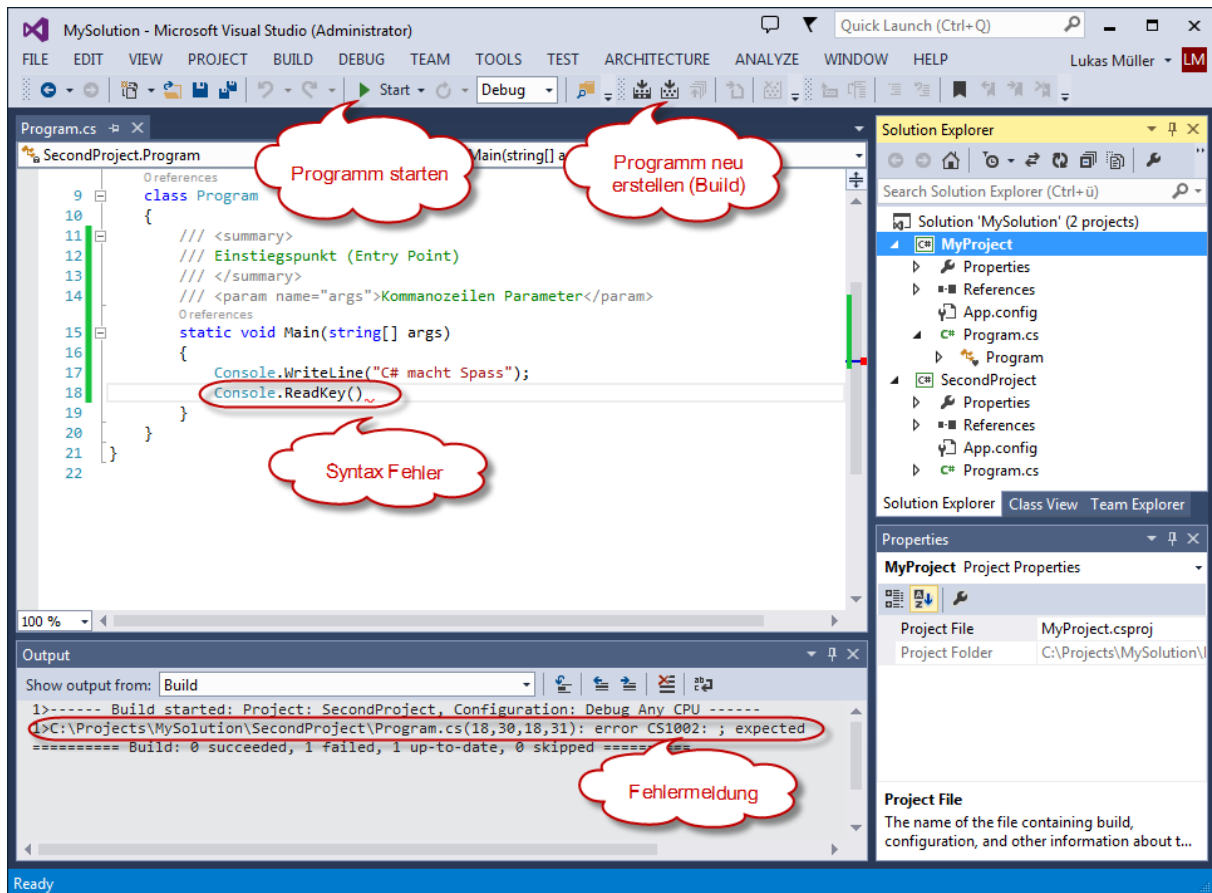


Besteht eine Projektmappe aus mehreren Projekten muss ein Projekt zum StartUp Projekt (Fettschrift) gewählt werden.



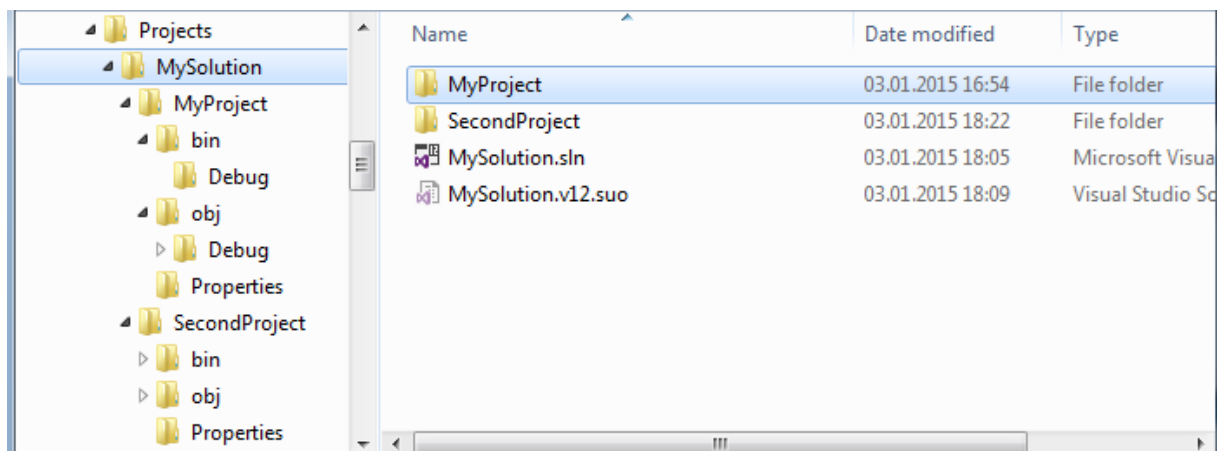


Programm neu erstellen (F7) und ausführen (F5).



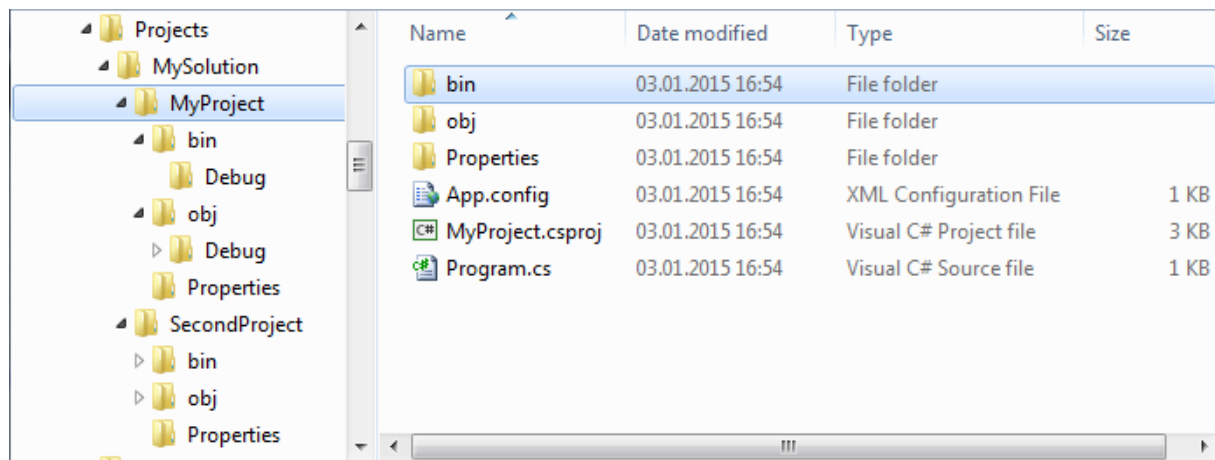
Ablagestruktur der Projektmappe im Dateisystem

In dem als Speicherort angegebenen Ordner der Projektmappe wird für jedes Projekt ein Unterordner mit dem Namen des Projektes angelegt. Zudem wird die Projektmappe mit Dateieindung `.sln` (Solution) abgespeichert.





Im Projektordner werden weiterhin die Code (.cs), Config, XML und die Projektdatei (.csproj) abgelegt. Die ausführbare .EXE Datei wird jeweils in der debug oder release Version unter dem bin Ordner erstellt.





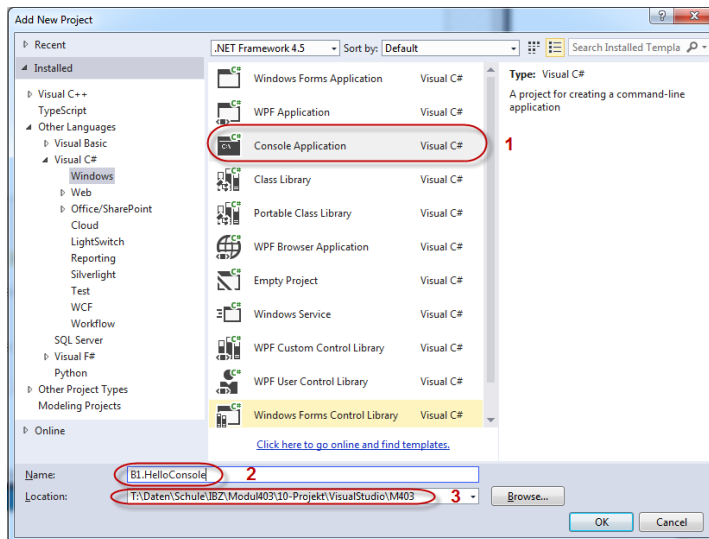
3.2. EINFÜHRUNG ENTWICKLUNGSUMGEBUNG

Folgen Sie den Anweisungen der Lehrperson und machen Sie sich allenfalls Notizen. Das Ziel ist, dass sie eine kleine Anwendung (Konsolenanwendung und Windowsanwendung) erstellen können, die einen Text auf dem Bildschirm ausgibt.

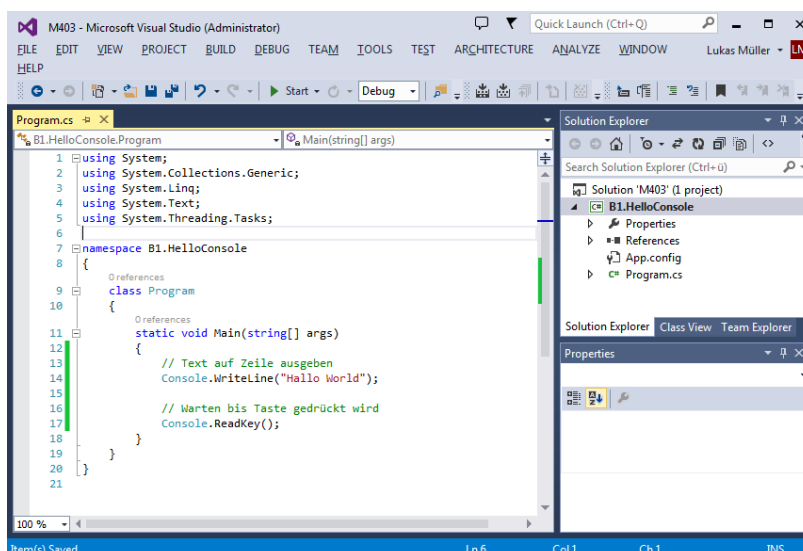
3.2.1. ERSTELLEN SIE EINE ERSTE C#-KONSOLENANWENDUNG

Starten Sie die VisualStudio.Net Entwicklungsumgebung. Wählen Sie den Menüpunkt Datei→Neu→Projekt.

Selektieren Sie im darauf erscheinenden Dialog den Eintrag Konsolenanwendung (siehe 1) und setzen Sie den Speicherortpfad auf c:\temp\ (siehe 3). Klicken Sie auf <OK>.



Erweitern Sie den Programmiercode die Methode Main() in der Program.cs Datei mit den unten aufgeführten Zeilen.



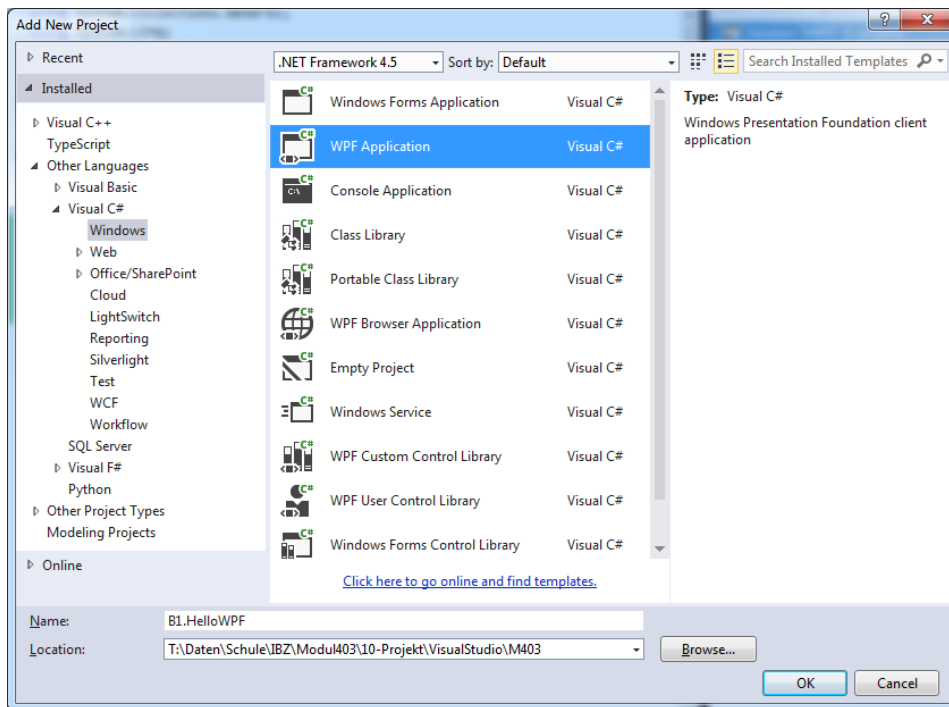
Kompilieren Sie Ihr Projekt (mittels Menüpunkt: Erstellen→Projektmappe erstellen) und führen Sie es aus (mittels Menüpunkt Debuggen→Starten).

Tipp: Sollte Ihre Applikation nur kurz erscheinen und danach gleich wieder verschwinden, können sie dies mittels der folgenden Anweisung, die am Schluss einzufügen ist, korrigieren:

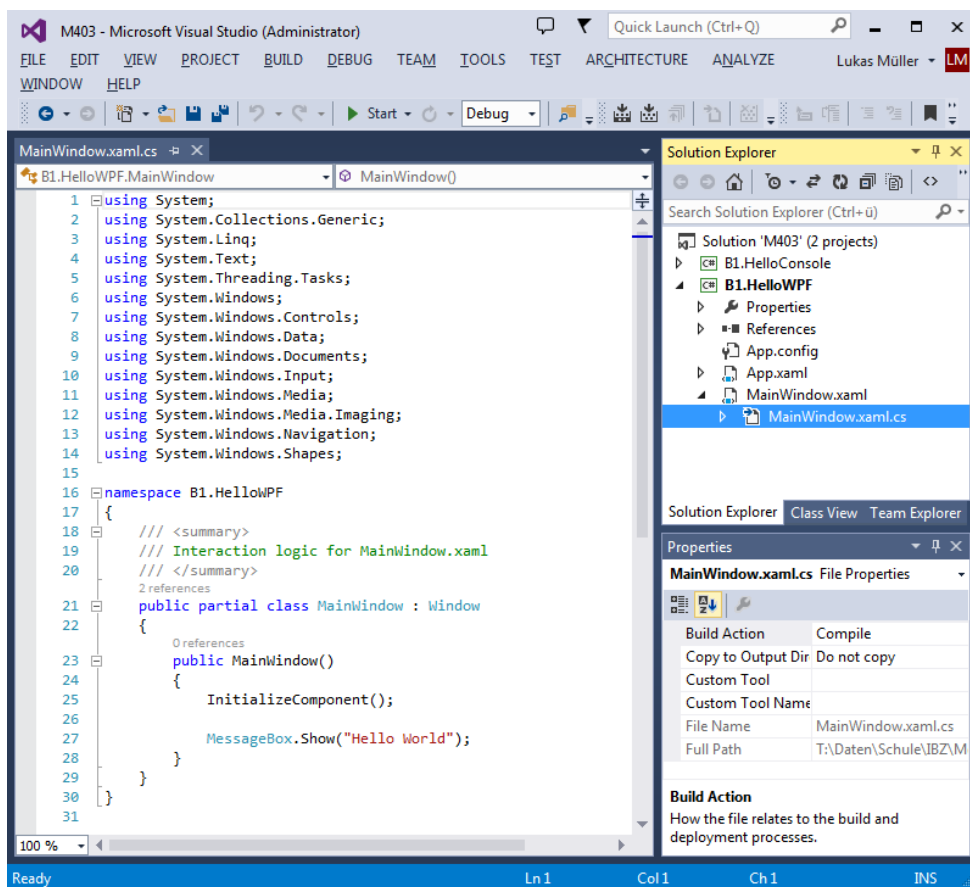
```
Console.ReadLine();    //Wartet auf eine Eingabe <Enter>
```




3.2.2. ERSTELLEN SIE EINE ERSTE C#-WPF ANWENDUNG



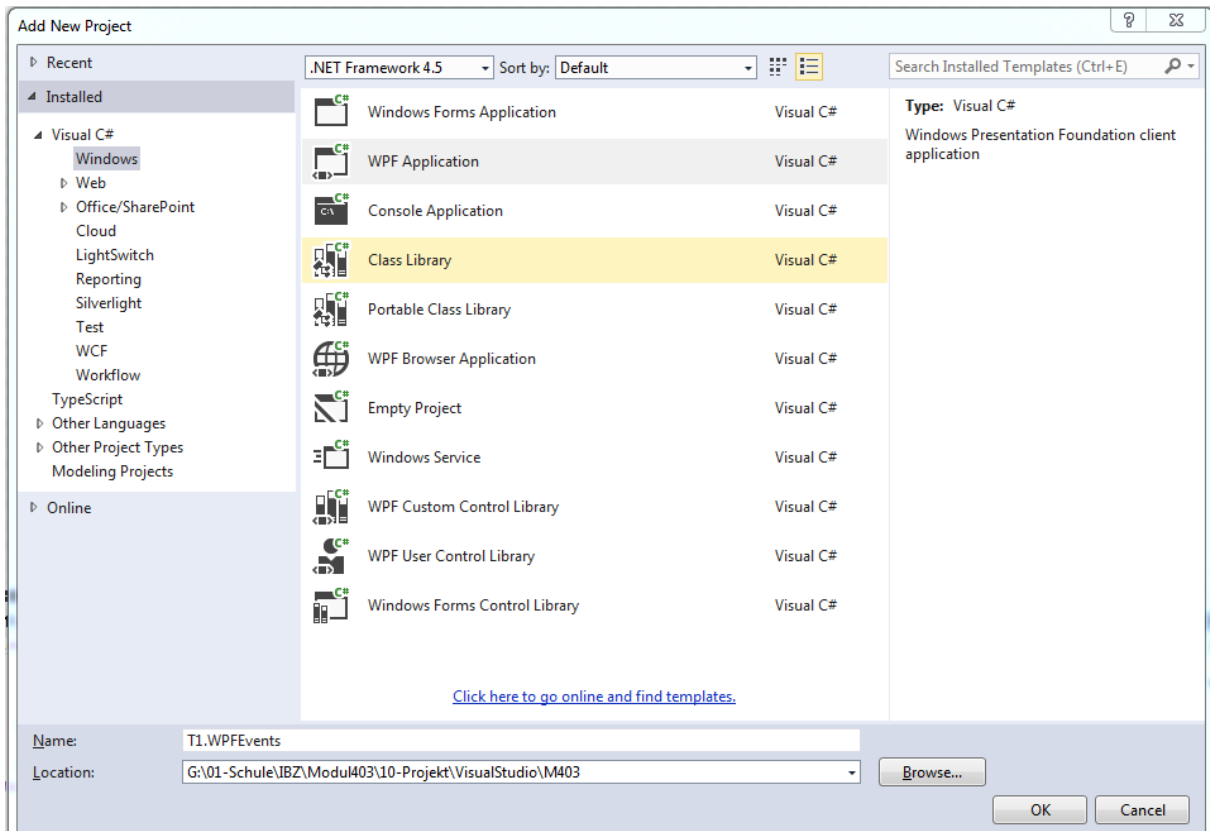
Fügen Sie der Methode `MainWindow()` in der Datei `MainWindow.xaml.cs` die untenstehende `MessageBox()` Zeile hinzu und starten Sie das Projekt mit `F5`.



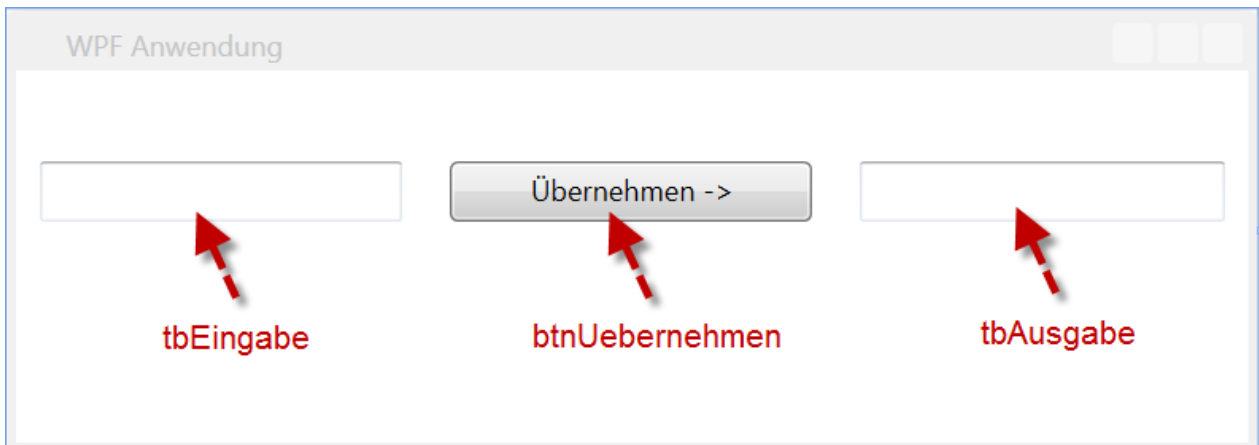


3.2.3. ERSTELLEN EINER C#-WINDOWS WPF ANWENDUNG:

Wählen Sie in der VisualStudio.Net Entwicklungsumgebung den Menüpunkt Datei→ Neu→Projekt.



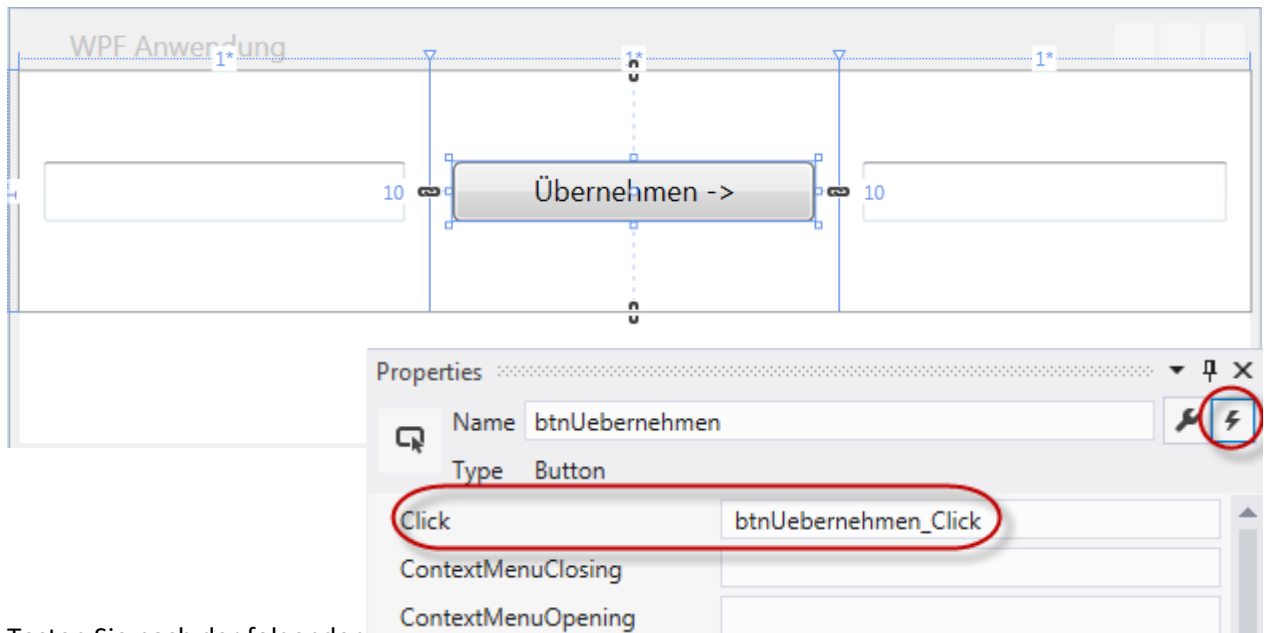
Ändern Sie danach Ihr Fensterlayout wie folgt ab



Setzen Sie die Eigenschaften der Kontrollelemente auf sinnvolle Bezeichnungen (z.B. tbEingabe). Das Eigenschaftsdialogfenster können Sie mittels KontextMenü des entspr. Steuerelementes aufrufen.



Teilen Sie dem Button mit, dass er auf ein Click-Ereignis reagieren soll. Öffnen Sie dazu den Eigenschaftsdialog des Button (siehe 1). Wählen Sie die Ansicht Ereignisse (siehe 2) und geben Sie eine "sprechende" Bezeichnung für das entspr. Ereignis ein (siehe 3). Wenn Sie <Enter> drücken wird der Code generiert.



Testen Sie nach der folgenden Codeänderung im Programm.

```
private void btnUebernehmen_Click(object sender, RoutedEventArgs e)
{
    string eingabe = tbEingabe.Text;

    if (eingabe != "")
    {
        tbAusgabe.Text = eingabe;
    }
}
```